Support Vector Machines

Python Programming and Investigation Problems

Industry Applications

General instructions

- Submit a Jupyter notebook containing code, figures, results, and written interpretations.
- Use NumPy, pandas, Matplotlib, and scikit-learn unless a problem specifies otherwise.
- Set random seeds to make experiments reproducible.
- Fit preprocessing transformations using training data only.
- Use validation data or cross-validation for model selection. Use the final test set only after selecting the complete procedure.
- Explain observed results. Do not submit plots or scores without interpretation.

1. Implement a linear SVM from scratch

Generate 300 two-dimensional observations using `make_blobs`, with two classes, cluster standard deviation 1.5, and random seed 42. Convert labels to $\{-1, +1\}$.

Use a stratified 70:30 training–test split and standardize the inputs using training statistics.

Implement the objective

$$J(w, b) = \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i=1}^{n} \max\{0, 1 - y_i(w^\top x_i + b)\}.$$

(a) Implement decision scores, predictions, hinge loss, and the complete objective using NumPy.

(b) Implement batch subgradient descent without using scikit-learn SVM training functions. At margin exactly 1, choose zero as the hinge-loss subgradient. Do not regularize b.

(c) Train with $\lambda = 0.01$ and investigate at least three learning rates.

(d) Plot the objective across iterations.

(e) Plot the learned boundary and canonical margin lines in standardized feature coordinates.

(f) Identify observations whose signed functional margin is at most 1, distinguishing those inside the margin from those on it.

(g) Compare with `SVC(kernel="linear")` using

$$C = \frac{1}{n\lambda},$$

where n is the training-set size.

(h) Explain differences that may arise from incomplete convergence and solver tolerances.

Required output: Training curves, boundary plots, test metrics, and a convergence discussion.

2. Investigate the effect of the penalty parameter

Generate 500 observations with `make_classification`: two informative features, no redundant features, one cluster per class, `class_sep=0.8`, `flip_y=0.08`, and random seed 42.

Create stratified training, validation, and test partitions.

(a) Train linear SVMs using

$$C \in \{0.001, 0.01, 0.1, 1, 10, 100, 1000\}.$$

(b) For each model, calculate training accuracy, validation accuracy, weight norm, margin width, and support-vector count.

(c) Calculate the number of training observations with $y_i f(x_i) < 1$ and the total training hinge loss.

(d) Plot representative decision boundaries.

(e) Select C using validation performance and report the selected model's test performance.

(f) Determine whether a larger C always produces fewer support vectors or better validation accuracy in your results.

(g) Explain why the geometric margin and training misclassification count measure different properties.

Required output: A comparison table, parameter plots, boundary plots, and a justified choice of C.

3. Investigate the interaction between C and γ

Generate 600 observations using `make_moons`, with noise 0.25 and random seed 42. Reserve a stratified 25% test set.

(a) Construct a pipeline containing standardization and an RBF SVM.

(b) Use five-fold stratified cross-validation on the remaining data to investigate

$$C \in \{0.01, 0.1, 1, 10, 100\},$$

$$\gamma \in \{0.01, 0.1, 1, 10, 100\}.$$

(c) Produce heatmaps of mean training and validation accuracy.

(d) Plot boundaries for configurations illustrating underfitting, reasonable generalization, and overfitting.

(e) Record support-vector counts for these configurations after fitting them on the full training partition.

(f) Select the final configuration using cross-validation and evaluate it on the test set.

(g) Explain why changing C cannot always compensate for an unsuitable γ.

Required output: Heatmaps, boundary plots, test metrics, and a joint parameter interpretation.

4. Match kernels to data geometry

Create three datasets, each containing 500 observations:

- Approximately linearly separable blobs.
- A noisy XOR dataset.
- Concentric circles using `make_circles`.

For XOR, sample coordinates uniformly from $[-1, 1]^2$, assign class 1 when $x_1 x_2 > 0$ and class 0 otherwise, then randomly flip 5% of the labels.

(a) Compare linear, degree-2 polynomial, and RBF SVMs.

(b) Use separate training and test partitions for each dataset. Tune parameters through cross-validation within training data.

(c) Plot decision boundaries and report test scores for all nine dataset–kernel combinations.

(d) Add x_1x_2 as an explicit feature to the XOR data and train a linear SVM.

(e) Compare the engineered-feature model with the polynomial-kernel model.

(f) Explain why nonlinear classification can arise from a linear separator in a transformed feature space.

Required output: Nine boundary plots, a performance table, and a feature-mapping explanation.

5. Which observations influence the boundary?

Generate two well-separated two-dimensional clusters with 200 observations and train a hard-margin-like linear SVM using a sufficiently large finite C.

Standardize the data once and keep those coordinates fixed throughout the experiment.

(a) Plot the boundary, margin lines, and support vectors.

(b) Remove one observation strictly outside the margin and retrain.

(c) Remove one support vector and retrain separately.

(d) Move a support vector progressively toward the opposite class and retrain after each movement.

(e) Compare the weight vector, intercept, support-vector set, and predictions on a fixed grid.

(f) Explain why removing a support vector need not always change the classifier.

(g) Explain why the effect of moving a point depends on whether it becomes influential after retraining.

Required output: Before-and-after plots and numerical comparisons of the fitted classifiers.

6. Feature scaling in an industrial sensor classifier

Generate a two-feature classification dataset. Multiply its second feature by 10,000 to simulate measurements expressed on very different numerical scales.

(a) Compare unscaled and standardized linear SVMs.

(b) Compare unscaled and standardized RBF SVMs.

(c) Perform a controlled comparison using identical hyperparameter settings within each kernel type.

(d) Then tune each preprocessing–model pipeline separately using cross-validation.

(e) Report predictive performance and support-vector counts.

(f) Plot boundaries in the original measurement coordinates.

(g) Explain how scaling affects RBF distances and the regularization geometry of a linear SVM.

(h) Demonstrate the effect of serving a fitted SVM with an inconsistent preprocessing transformation.

Required output: Controlled and tuned comparisons, boundary plots, and a preprocessing consistency discussion.

7. Detect rare manufacturing defects

Generate 5,000 observations using `make_classification`, with 12 features, six informative features, two redundant features, class weights $[0.97, 0.03]$, `flip_y=0.01`, and random seed 42. Treat class 1 as defective.

(a) Create stratified training, validation, and test partitions.

(b) Establish the performance of an always-normal baseline.

(c) Compare an unweighted SVM with `class_weight="balanced"` and at least three custom positive-class weights.

(d) Keep the kernel and C fixed during the initial class-weight comparison.

(e) Report confusion matrices, defect precision, defect recall, F1 score, and average precision.

(f) Plot precision–recall curves using decision scores.

(g) Select a model using a stated operational objective.

(h) Explain why overall accuracy can favour a detector that misses most defects.

Required output: Metric tables, confusion matrices, precision–recall curves, and an operational recommendation.

8. Choose a threshold under inspection constraints

Use a fitted defect classifier from Problem 7. Do not retrain it while comparing thresholds.

Assume:

- A missed defect costs 500 cost units.
- An unnecessary inspection of a normal item costs 10 units.
- At most 8% of items can be flagged for inspection.

(a) Obtain validation decision scores.

(b) For each candidate threshold t, predict a defect when $f(x) \geq t$.

(c) Calculate

$$\text{Cost}(t) = 500\,FN(t) + 10\,FP(t).$$

(d) Calculate the fraction of items flagged at each threshold.

(e) Select the minimum-cost validation threshold satisfying the inspection constraint.

(f) Compare it with the default threshold $t = 0$.

(g) Apply the selected threshold unchanged to the test set.

(h) Report whether the inspection constraint still holds on the test set and explain why it is not guaranteed to do so.

Required output: Cost and inspection-rate plots, selected threshold, and test operating results.

9. Decision scores versus calibrated probabilities

Use an imbalanced binary dataset with at least 3,000 observations. Divide it into model-training, calibration, and test partitions.

(a) Fit a standardized SVM using only the model-training partition.

(b) Obtain its decision scores on the calibration partition.

(c) Fit a sigmoid calibration model using those scores and calibration labels.

(d) Compare raw scores and calibrated probabilities on the test partition.

(e) Plot a reliability diagram and calculate Brier score and log loss for the calibrated probabilities.

(f) For comparison, treat the transformation

$$p(x) = \frac{1}{1 + \exp[-f(x)]}$$

as an uncalibrated probability estimate and evaluate it using the same probability metrics.

(g) Compare ranking performance using ROC AUC and average precision.

(h) Explain why improved probability calibration need not imply improved ranking or higher classification accuracy.

Required output: Reliability diagram, probability metrics, ranking metrics, and interpretation.

10. One-class SVM for machine anomaly detection

Construct two-dimensional synthetic observations as follows:

- Training normal operation: 1,000 observations from $\mathcal{N}((0,0)^\top, I)$.
- Validation and test sets: independent mixtures of known normal operation, fault observations from $\mathcal{N}((4,4)^\top, 0.5I)$, and an unseen safe operating regime from $\mathcal{N}((-4,0)^\top, 0.5I)$.

For operational evaluation, both normal regimes are safe; only fault observations are actual anomalies.

(a) Fit the scaler using training normal data only.

(b) Train one-class SVMs for several ν and γ values.

(c) Evaluate fault recall and false-alarm rates separately for the two safe regimes.

(d) Plot the learned acceptance regions.

(e) Select a configuration using validation data.

(f) Add representative observations from the unseen safe regime to the normal training data and retrain.

(g) Repeat the evaluation on untouched test observations.

(h) Explain the distinction between novelty detection and fault detection.

(i) Investigate the effect of contaminating normal training data with 5% fault observations.

Required output: Acceptance-region plots, regime-specific metrics, and a contamination analysis.

11. SVR: tolerance, fit, and sparsity

Generate 400 observations with

$$x \sim \text{Uniform}(0, 10),$$

$$y = \sin(x) + 0.2x + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 0.15^2).$$

Create training, validation, and test partitions. Standardize inputs, but keep targets in their original units.

(a) Train RBF SVR models for
$$\varepsilon \in \{0.01, 0.1, 0.3, 0.6\}$$
using a fixed C and γ.

(b) Plot each regression curve and its ε-insensitive tube.

(c) Record support-vector counts, validation MAE, validation RMSE, and training epsilon-insensitive loss.

(d) Repeat the comparison for several values of C.

(e) Select a configuration using validation performance and evaluate it on the test set.

(f) Compare predictions inside the training input range with predictions beyond that range.

(g) Explain why widening the tube changes the training tolerance but does not guarantee a particular test error.

Required output: Regression and tube plots, a parameter-comparison table, and an extrapolation discussion.

12. Predictive performance versus deployment cost

Generate a nonlinear classification dataset with `make_moons`, using at least 5,000 observations. Use fixed training and test partitions.

Compare:

- A linear SVM.
- An exact RBF SVM.
- A linear SVM trained on approximate RBF features using `RBFSampler` or `Nystroem`.

(a) Tune each complete pipeline using training data only.

(b) Compare test accuracy, F1 score, and average precision.

(c) Measure training time and repeated batch prediction time. Include preprocessing and feature-map computation in timing.

(d) Report model storage size and support-vector count where applicable.

(e) Repeat approximate-kernel training for several feature-map sizes.

(f) Repeat timing measurements and report a summary rather than relying on one execution.

(g) Select a model for a deployment requirement you state, such as a memory budget or batch prediction deadline.

(h) Explain why a finite feature approximation trades representation accuracy against computational cost.

Required output: Performance, timing, and storage tables, plus a deployment recommendation.

Integrated Industry Project

Project: Design a manufacturing defect detection system
A factory wants to identify rare defective products from sensor measurements. Missed defects are expensive, inspection capacity is limited, and products are manufactured in batches whose measurement distributions differ slightly.
Dataset construction
Using NumPy:

- Generate 10,000 observations with 12 sensor features and assign them to 50 batches of 200 observations each.
- Generate a latent feature vector $z_i \sim \mathcal{N}(0, I_{12})$ for each observation.
- For each batch, generate a sensor offset $\delta_b \sim \mathcal{N}(0, 0.3^2 I_{12})$. Define observed measurements as $x_i = z_i + \delta_b$.
- Define a latent defect-risk score

$$r_i = z_{i1}z_{i2} + 0.8z_{i3} + 0.5z_{i4}^2 + \eta_i, \qquad \eta_i \sim \mathcal{N}(0, 0.3^2).$$

- Assign the positive class to observations above the 97th percentile of the generated risk scores.
- Randomly flip 0.5% of the labels.
- Rescale selected sensor features by factors of 100, 1,000, and 10,000.

This construction is a synthetic teaching benchmark. Treat the risk scores and latent variables as unavailable to the deployed classifier.
Tasks

(a) Reserve complete batches for the final test set. Use batch-aware validation within the remaining batches.

(b) Explain why random observation-level splitting may give a different estimate from batch-aware evaluation.

(c) Build preprocessing pipelines and establish a simple classification baseline.

(d) Compare linear, polynomial, and RBF SVMs.

(e) Tune hyperparameters and investigate class weighting.

(f) Select a decision threshold using validation predictions, assuming a missed-defect cost of 500 units, a false-alarm cost of 10 units, and a maximum inspection rate of 8%.

(g) Evaluate the selected pipeline once on the held-out batches. Report defect precision, recall, F1 score, average precision, confusion matrix, inspection rate, and cost per 1,000 products.

(h) Measure prediction time and model storage requirements.

(i) Investigate performance on a separate stress-test set with larger batch offsets.

(j) State which model you recommend, why you recommend it, and which operating conditions require further data.

Submission: One reproducible Jupyter notebook and a brief report covering data preparation, evaluation design, model selection, operating threshold, deployment cost, and limitations.